

Punto 3.8 — Eager Loading con Include y ThenInclude

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.7. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Eager Loading con Include y ThenInclude** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.8
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerOrdenesConPlanchasInclude`, `ObtenerOrdenesConPlanchasYDetalleInclude`, `ObtenerOrdenesConAleacionesInclude`, `ObtenerOrdenesConPlanchasPesadasInclude`, `ObtenerOrdenesConPlanchasYDetalleSplitQuery`, `ObtenerSqlInclude`, `ObtenerOrdenesAutoInclude` y `ObtenerOrdenesIgnorandoAutoInclude`. El punto demuestra además la configuración `AutoInclude` y su neutralización por consulta mediante `IgnoreAutoIncludes`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaEagerUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CargaEagerUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaEagerUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== EAGER LOADING ===");
        var planchas = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();
        var detalle = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude();
        var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude();
        var filtradas = _unidad.Ordenes.ObtenerOrdenesConPlanchasPesadasInclude();
        var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();
        var auto = _unidad.Ordenes.ObtenerOrdenesAutoInclude();
        var sinAuto = _unidad.Ordenes.ObtenerOrdenesIgnorandoAutoInclude();

        if (planchas.Count != 5 || detalle.Count != 5 || aleaciones.Count != 5 || split.Count != 5)
            throw new InvalidOperationException("Carga Eager inesperada.");
        if (filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 1)
            throw new InvalidOperationException("Filtered Include inesperado.");
        if (auto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 2)
            throw new InvalidOperationException("AutoInclude no cargó Planchas.");
        if (sinAuto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 0)
            throw new InvalidOperationException("IgnoreAutoIncludes no suprimió la carga automática.");
        if (split.Sum(o => o.Planchas.Count) != 5 || split.Sum(o => o.OrdenesAleaciones.Count) != 4)
            throw new InvalidOperationException("SplitQuery no materializó las dos colecciones
esperadas.");

        Console.WriteLine($"Include: {planchas.Count} órdenes | SplitQuery: {split.Count} | AutoInclude:
{auto.Count} | IgnoreAutoIncludes: {sinAuto.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlInclude());
    }
}

```

Explicación línea a línea del caso de uso 3.8

Línea 1: `using AceriaData.Application.Interfaces;` → Importa `IUnidadDeTrabajo`, el puerto que permite al caso de uso consumir consultas sin conocer `AceriaDbContext`.

Línea 3: `namespace AceriaData.Application.UseCases;` → Sitúa la clase en el espacio de nombres de casos de uso de `Application`.

Línea 5: `public sealed class CargaEagerUseCase` → Declara el caso de uso dedicado a comparar las variantes de Eager Loading del checkpoint.

Línea 6: `{` → Abre el bloque de la declaración inmediatamente anterior.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Conserva la unidad de trabajo inyectada para acceder al repositorio de órdenes.

Línea 8: `public CargaEagerUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante DI y la asigna al campo de solo lectura.

Línea 10: `public void Ejecutar()` → Define la operación que ejecutará todas las comprobaciones E2E de carga Eager.

Línea 11: `{` → Abre el bloque de la declaración inmediatamente anterior.

Línea 12: `Console.WriteLine("=== EAGER LOADING ===");` → Muestra la cabecera de la demostración.

Línea 13: `var planchas = _unidad.Ordenes.ObtenerOrdenesConPlanchasInclude();` → Ejecuta `Include` sobre la colección `Planchas`.

Línea 14: `var detalle = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleInclude();` → Carga conjuntamente Planchas y la referencia Detalle.

Línea 15: `var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleacionesInclude();` → Ejecuta Include + ThenInclude sobre OrdenesAleaciones → Aleacion.

Línea 16: `var filtradas = _unidad.Ordenes.ObtenerOrdenesConPlanchasPesadasInclude();` → Ejecuta Filtered Include y conserva las planchas que cumplen el umbral del repositorio.

Línea 17: `var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSplitQuery();` → Ejecuta AsSplitQuery sobre Planchas y OrdenesAleaciones, más Detalle, para evitar expansión cartesiana entre colecciones hermanas.

Línea 18: `var auto = _unidad.Ordenes.ObtenerOrdenesAutoInclude();` → Consulta órdenes sin Include explícito; el modelo debe cargar Planchas automáticamente mediante AutoInclude.

Línea 19: `var sinAuto = _unidad.Ordenes.ObtenerOrdenesIgnorandoAutoInclude();` → Ejecuta la lectura con IgnoreAutoIncludes para demostrar que la carga automática puede suprimirse por consulta.

Línea 21: `if (planchas.Count != 5 || detalle.Count != 5 || aleaciones.Count != 5 || split.Count != 5)` → Comprueba que todas las estrategias devuelven las cinco órdenes visibles del dataset.

Línea 22: `throw new InvalidOperationException("Carga Eager inesperada.");` → Hace fallar el E2E si la forma principal de la carga no coincide con el dataset.

Línea 23: `if (filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 1)` → Verifica que Filtered Include deja una sola plancha de al menos 300 kg en OF-2024-0001.

Línea 24: `throw new InvalidOperationException("Filtered Include inesperado.");` → Falla si el filtro de la colección incluida no se respeta.

Línea 25: `if (auto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 2)` → Comprueba que AutoInclude cargó las dos planchas de OF-2024-0001 sin Include explícito.

Línea 26: `throw new InvalidOperationException("AutoInclude no cargó Planchas.");` → Falla si la navegación configurada con AutoInclude no fue materializada.

Línea 27: `if (sinAuto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count != 0)` → Comprueba que IgnoreAutoIncludes deja la colección sin cargar en una consulta AsNoTracking.

Línea 28: `throw new InvalidOperationException("IgnoreAutoIncludes no suprimió la carga automática.");` → Falla si la consulta no logra neutralizar AutoInclude.

Línea 29: `if (split.Sum(o => o.Planchas.Count) != 5 || split.Sum(o => o.OrdenesAleaciones.Count) != 4)` → Valida que SplitQuery materializó las cinco planchas y las cuatro relaciones de aleación del dataset.

Línea 30: `throw new InvalidOperationException("SplitQuery no materializó las dos colecciones esperadas.");` → Falla si alguna de las dos colecciones incluidas queda incompleta.

Línea 32: `Console.WriteLine($"Include: {planchas.Count} órdenes | SplitQuery: {split.Count} | AutoInclude: {auto.Count} | IgnoreAutoIncludes: {sinAuto.Count}");` → Resume en consola los tamaños obtenidos por las estrategias validadas.

Línea 33: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlInclude());` → Imprime el SQL generado para la consulta Include de referencia.

Línea 34: `}` → Cierra el bloque de la declaración inmediatamente anterior.

Línea 35: `}` → Cierra el bloque de la declaración inmediatamente anterior.

▮ Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```

using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaEagerUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.8 OK");

```

Explicación línea a línea de Program.cs 3.8

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<CargaEagerUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<CargaEagerUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.8 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.8 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.8, el Filtered Include usa `AsNoTracking()` para que navigation fix-up no reincorpore planchas previamente rastreadas que no cumplan el filtro.

Paso 9: Diagnosticar un error común

Error: Filtered Include con entidades ya rastreadas puede verse afectado por navigation fix-up; el ejemplo usa `AsNoTracking`.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.8 - METODOS PEDAGOGICOS EAGER

```
// Se activan sólo en copias desechables para demostrar fix-up y comparar Single/Split.
int ObtenerCantidadPlanchasFilteredIncludeConTrackingReto();
string ObtenerSqlCargaCompletaSingleQueryReto();
string ObtenerSqlCargaCompletaSplitQueryReto();
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.8 - METODOS PEDAGOGICOS EAGER

```
// Error controlado: primero rastrea el grafo completo y después aplica Filtered Include
// en el mismo DbContext. Navigation fix-up conserva las dos planchas rastreadas.
public int ObtenerCantidadPlanchasFilteredIncludeConTrackingReto()
{
    _context.ChangeTracker.Clear();

    _ = _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .Include(o => o.Planchas)
        .Single(o => o.NumeroOrden == "OF-2024-0001");

    var filtrada = _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .Include(o => o.Planchas.Where(p => p.Peso >= 300m))
        .Single(o => o.NumeroOrden == "OF-2024-0001");

    return filtrada.Planchas.Count;
}

public string ObtenerSqlCargaCompletaSingleQueryReto() => _context.OrdenesFabricacion
    .IgnoreAutoIncludes().AsNoTracking()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones).ThenInclude(oa => oa.Aleacion)
    .Include(o => o.Detalle)
    .AsSingleQuery()
    .OrderBy(o => o.NumeroOrden)
    .ToString();

public string ObtenerSqlCargaCompletaSplitQueryReto() => _context.OrdenesFabricacion
    .IgnoreAutoIncludes().AsNoTracking()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones).ThenInclude(oa => oa.Aleacion)
    .Include(o => o.Detalle)
    .AsSplitQuery()
    .OrderBy(o => o.NumeroOrden)
    .ToString();
```

Archivo: src/AceriaData.Application/CargaEagerUseCase.cs

Marcador: ERROR CONTROLADO M03 3.8 - FILTERED INCLUDE CON TRACKING Y FIX-UP

```
// Demuestra el Paso 9: entidades ya rastreadas pueden reincorporarse a la navegación filtrada.
var conTracking = _unidad.Ordenes.ObtenerCantidadPlanchasFilteredIncludeConTrackingReto();
var segura = filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count;

if (conTracking != 2 || segura != 1)
    throw new InvalidOperationException("Error controlado 3.8: no se reprodujo la diferencia tracking/fix-up frente a AsNoTracking.");

Console.WriteLine($"Error controlado 3.8 OK | Tracking: {conTracking} | AsNoTracking: {segura}");
```

Comprobación esperada: Filtered Include con tracking puede reincorporar entidades por navigation fix-up.

Laboratorio adicional del punto 3.8

Diagnóstico técnico

El checkpoint ejecuta `Include`, `ThenInclude`, `Filtered Include`, `AsSplitQuery()`, `AutoInclude()` e `IgnoreAutoIncludes()`. El split carga dos colecciones hermanas —Planchas y OrdenesAleaciones— más Detalle, de modo que la decisión responde a un riesgo real de multiplicación de filas.

Reto resuelto y verificación adicional

Valida OF-2024-0001 con dos planchas mediante `AutoInclude`, cero planchas usando `IgnoreAutoIncludes` + `AsNoTracking`, una sola plancha en `Filtered Include` y, en `SplitQuery`, cinco planchas y cuatro relaciones de aleación en todo el dataset.

Errores comunes revisados

Error	Causa	Solución
Multiplicación de filas	Se cargaron colecciones hermanas al mismo nivel en modo single-query	Comparar <code>AsSingleQuery()</code> y <code>AsSplitQuery()</code> para el volumen real
N+1	Se accedió repetidamente a navegaciones Lazy no cargadas	Planificar Eager Loading, proyección o Explicit Loading según el caso
ThenInclude sin Include	Se usó ThenInclude sin Include previo	Usar Include antes de ThenInclude
NullReferenceException en referencia	La propiedad de navegación es null	Comprobar si la propiedad es null
Filtro no aplicado	Se aplicó el filtro fuera del Include	Aplicar el filtro dentro del Include
AutoInclude excesivo	Se configuró en muchas propiedades	Usar con moderación

Analogía operativa

La carga Eager en una acería es como pedirle al archivo central que, además de la carpeta de la orden, adjunte también las carpetas de las planchas y del detalle. En lugar de pedir la carpeta de la orden y después ir a buscar las planchas y el detalle por separado, se pide todo junto. El archivo central prepara un paquete con la orden y sus documentos relacionados. Include es como pedir que se adjunte una carpeta. ThenInclude es como pedir que se adjunten los documentos de la carpeta adjunta. Filtered Include es como pedir que solo se adjunten las planchas activas. AsSplitQuery es como pedir que el paquete se prepare en varios envíos separados para evitar que el paquete sea demasiado grande. AutoInclude es como pedir que siempre se adjunten las planchas, sin tener que pedirlo cada vez. Así funciona la carga Eager en EF Core: las relaciones necesarias se solicitan de forma planificada como parte de la operación inicial, que puede resolverse con una consulta única o con varias consultas coordinadas mediante `AsSplitQuery()`.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Compara Include único y AsSplitQuery con las mismas relaciones y observa el SQL.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: APOYO M03 3.8 - METODOS PEDAGOGICOS EAGER

```
// Se activan sólo en copias desechables para demostrar fix-up y comparar Single/Split.
int ObtenerCantidadPlanchasFilteredIncludeConTrackingReto();
string ObtenerSqlCargaCompletaSingleQueryReto();
string ObtenerSqlCargaCompletaSplitQueryReto();
```

Archivo: `src/AceriaData.Infrastructure/Repositories/Repositories.cs`

Marcador: APOYO M03 3.8 - METODOS PEDAGOGICOS EAGER

```
// Error controlado: primero rastrea el grafo completo y después aplica Filtered Include
// en el mismo DbContext. Navigation fix-up conserva las dos planchas rastreadas.
public int ObtenerCantidadPlanchasFilteredIncludeConTrackingReto()
{
    _context.ChangeTracker.Clear();

    _ = _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .Include(o => o.Planchas)
        .Single(o => o.NumeroOrden == "OF-2024-0001");

    var filtrada = _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .Include(o => o.Planchas.Where(p => p.Peso >= 300m))
        .Single(o => o.NumeroOrden == "OF-2024-0001");

    return filtrada.Planchas.Count;
}

public string ObtenerSqlCargaCompletaSingleQueryReto() => _context.OrdenesFabricacion
    .IgnoreAutoIncludes().AsNoTracking()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones).ThenInclude(oa => oa.Aleacion)
    .Include(o => o.Detalle)
    .AsSingleQuery()
    .OrderBy(o => o.NumeroOrden)
    .ToString();

public string ObtenerSqlCargaCompletaSplitQueryReto() => _context.OrdenesFabricacion
    .IgnoreAutoIncludes().AsNoTracking()
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones).ThenInclude(oa => oa.Aleacion)
    .Include(o => o.Detalle)
    .AsSplitQuery()
    .OrderBy(o => o.NumeroOrden)
    .ToString();
```

Archivo: src/AceriaData.Application/CargaEagerUseCase.cs

Marcador: RETO M03 3.8 - SINGLEQUERY VS SPLITQUERY MISMO GRAFO

```
// Demuestra el Paso 10 y el laboratorio adicional con las mismas relaciones.
var autoReto = auto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count;
var sinAutoReto = sinAuto.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count;
var filtradaReto = filtradas.Single(o => o.NumeroOrden == "OF-2024-0001").Planchas.Count;
var splitPlanchas = split.Sum(o => o.Planchas.Count);
var splitAleaciones = split.Sum(o => o.OrdenesAleaciones.Count);

if (autoReto != 2 || sinAutoReto != 0 || filtradaReto != 1 ||
    splitPlanchas != 5 || splitAleaciones != 4)
    throw new InvalidOperationException("Reto 3.8: el grafo cargado no coincide con el dataset
esperado.");

var sqlSingle = _unidad.Ordenes.ObtenerSqlCargaCompletaSingleQueryReto();
var sqlSplit = _unidad.Ordenes.ObtenerSqlCargaCompletaSplitQueryReto();

Console.WriteLine($"Reto 3.8 OK | Auto: {autoReto} | SinAuto: {sinAutoReto} | Filtrada:
{filtradaReto} | Split Planchas: {splitPlanchas} | Split Aleaciones: {splitAleaciones}");
Console.WriteLine("RETO_SINGLE_SQL_INICIO");
Console.WriteLine(sqlSingle);
Console.WriteLine("RETO_SINGLE_SQL_FIN");
Console.WriteLine("RETO_SPLIT_SQL_INICIO");
Console.WriteLine(sqlSplit);
Console.WriteLine("RETO_SPLIT_SQL_FIN");
```

Comprobación esperada: AsSingleQuery y AsSplitQuery cargan el mismo grafo y permiten comparar el SQL.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.8 OK, el siguiente estado parte exactamente de esta solución y añade **Lazy Loading: configuración, funcionamiento y riesgos**.
